

Dockerized ROS2 EuRoC Web Calibration Lab

Part 1: Docker and ROS2 essentials - Part 2: Image capture and undistortion

Advanced Mobility Course

50-minute hands-on session

Lab deliverable

Replay a ROS2 camera bag, view the camera stream in a browser, capture a frame, and apply calibration to produce an undistorted comparison image.

Why this lab follows the calibration lecture

Monday

Camera calibration estimates numerical parameters:

- intrinsic matrix K
- distortion coefficients D
- camera geometry model

Wednesday

We use those parameters in a robotics pipeline:

- ROS2 camera topic
- web visualization
- frame capture
- undistortion by remapping

Calibration becomes an operator on image streams, not just a table of numbers.

50-minute session map

Part	Time	Main question	Student output
1	25 min	How do Docker and ROS2 make the lab reproducible?	Running container, ROS2 topics, bag re-play
2	25 min	How does calibration change the image stream?	Raw/undistorted image pair with metadata

Success criterion

The browser displays the camera stream at `localhost:8080`; a captured raw frame and its undistorted result are saved side by side.

Part 1 learning objectives

By the end of Part 1, students should be able to explain:

- 1 why Docker is useful for robotics software labs;
- 2 what image, container, volume, port, and Compose mean;
- 3 how ROS2 represents a robot system as a graph;
- 4 how to inspect camera topics with ROS2 CLI commands;
- 5 how `ros2 bag play` replays sensor logs.

Scope control

Do not teach full Docker administration or full ROS2 package development here. Teach only what is needed to run and debug the lab.

Codex prompt for Docker installation

Use this prompt before students touch ROS2 locally.

You are helping me prepare a ROS2 Jazzy Docker-based lab environment.

Task:

1. Check whether Docker Desktop or Docker Engine is installed on my OS.
2. If not installed, guide me to install Docker using official Docker documentation only.
3. Verify Docker with:
 `docker --version`
 `docker run hello-world`
4. Do not install ROS2 directly on my host machine.
5. The final goal is to run a Docker Compose project for a ROS2 Jazzy EuRoC camera-streaming lab.

Constraints:

- Use official Docker documentation links.
- Do not use random blog posts.
- Stop and ask before making destructive system-level changes.

Official installation links

Docker

Docker Desktop is the recommended install path for most student laptops.

- Overview: docs.docker.com/desktop
- Get Docker Desktop: docs.docker.com/get-started
- Windows: Windows install guide
- Mac: Mac install guide
- Ubuntu: Ubuntu install guide

ROS2

Use ROS2 Jazzy in the container. Jazzy was released on May 23, 2024 and has EOL in May 2029.

Source: Docker Desktop docs; ROS2 Jazzy distribution page

Docker mental model for robotics labs

Concept	Plain meaning	Role in this lab
Image	Read-only environment template	ROS2 Jazzy, Python, OpenCV, web app
Container	Running process created from an image	The active lab machine
Volume	Host-container shared folder	Saved raw and undistorted images
Port	Network exposure from container to host	Browser opens localhost:8080
Compose	Single file to run the stack	Build, run, and configure the demo

One sentence

Docker turns “it worked on my laptop” into a controlled experiment environment.

Minimal Docker commands

```
# Build and start the full demo
docker compose up --build

# Stop containers
docker compose down

# Show runtime logs
docker compose logs -f

# Enter the ROS2 shell inside the container
docker compose run --rm ros2-web-lab shell
```

Debug rule

If the browser does not show images, first inspect Compose logs, then inspect ROS2 topics from inside the container.

ROS2 graph for this lab

Data flow

```
ros2 bag play → /cam0/image_raw, /cam0/camera_info → web_bridge_node → browser  
UI
```

ROS2 element	In this lab
--------------	-------------

Node	Bag player and image web bridge
Topic	/cam0/image_raw, /cam0/camera_info
Message	sensor_msgs/msg/Image, sensor_msgs/msg/CameraInfo
QoS	Sensor stream policy for latency and frame drops
Bag	Replayable sensor log for reproducible experiments

ROS2 CLI checklist

```
# Inside the container
source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash

ros2 topic list
ros2 topic info /cam0/image_raw
ros2 topic info /cam0/camera_info
ros2 topic hz /cam0/image_raw
ros2 bag info /data/euroc_mh01_subset
```

What students should observe

The image topic and camera-info topic are published by the bag player, then consumed by the web bridge.

Replay the sensor log

```
ros2 bag play /data/euroc_mh01_subset --loop --rate 0.5
```

Option	Meaning
/data/euroc_mh01_subset	ROS2 bag path inside the container
-loop	Replay repeatedly for classroom interaction
-rate 0.5	Slow down playback so students can capture frames

Source: ROS2 bag tutorial

Part 1 checkpoint

Students pass Part 1 if they can show three things

- 1 `ros2 topic list` includes `/cam0/image_raw`;
- 2 `ros2 topic hz /cam0/image_raw` reports a live stream;
- 3 `http://localhost:8080` displays the image stream.

Transition question

If `/cam0/image_raw` is only pixel data, what extra information is needed to undistort it?

Part 2 learning objectives

By the end of Part 2, students should be able to explain:

- ① what the EuRoC MAV dataset provides;
- ② why image frames should be stored with calibration metadata;
- ③ how a browser can subscribe indirectly to ROS2 data;
- ④ how radial-tangential distortion maps normalized points;
- ⑤ why undistortion is usually implemented as image remapping.

EuRoC MAV dataset in one slide

Dataset role

EuRoC contains visual-inertial data collected onboard a micro aerial vehicle. For this lab, we use only the cam0 image stream and calibration parameters.

Data type	Why it matters
Stereo images	Camera stream for visual odometry and calibration demos
IMU measurements	Future extension to visual-inertial odometry
Ground truth	Future extension to trajectory evaluation
Camera calibration	Intrinsics and distortion used today
Camera-IMU extrinsics	Future extension to sensor fusion

Source: [ETH ASL EuRoC MAV dataset page](#) and [ETH Research Collection](#)

Dataset preparation strategy

Mode	What happens	When to use
Default	Create a small EuRoC-like bag during Docker build	Live class reliability
Official	Download MH_01_easy ZIP and extract first N cam0 frames	Full dataset demonstration
Local	Convert an already extracted EuRoC folder to a ROS2 bag	Lab machines with preloaded data

Classroom recommendation

Use the default mode for live teaching. Use official mode before class if you want real EuRoC frames and the network can tolerate the download.

Web demo architecture

Runtime pipeline

ROS2 bag → camera topics → FastAPI bridge → MJPEG stream → browser

Saved experiment sample

A capture is not only an image. It is:

raw image + timestamp + topic + K + D + distortion model

Undistortion service

The browser calls an HTTP endpoint; the backend loads the raw image and metadata, applies OpenCV remapping, then saves the corrected image.

Why CameraInfo matters

Topic	Meaning
/cam0/image_raw	Distorted raw intensity image
/cam0/camera_info	Camera matrix, distortion coefficients, rectification, projection
/cam0/image_rect	Rectified or undistorted image, if another node produces it

Key point

Pixel values alone do not define the camera geometry. Calibration metadata is needed to map pixels back to camera rays.

Source: ROS2 sensor_msgs/CameraInfo documentation

User capture routine

- 1 The browser displays the live stream.
- 2 The student clicks **Capture Current Frame**.
- 3 Backend copies the latest ROS2 frame buffer.
- 4 Raw image is saved as `saved/raw/frame_XXXXXX.png`.
- 5 Metadata is saved as `saved/metadata/frame_XXXXXX.json`.
- 6 Thumbnail appears in the gallery.

Student mission

Capture a frame with straight structures near the image boundary. Boundary regions make distortion effects easier to see.

Saved metadata example

```
{
  "image_id": "frame_000012",
  "topic": "/cam0/image_raw",
  "timestamp_ros_ns": 1403636579763555584,
  "encoding": "mono8",
  "width": 752,
  "height": 480,
  "camera": "cam0",
  "K": [[458.654, 0.0, 367.215],
        [0.0, 457.296, 248.375],
        [0.0, 0.0, 1.0]],
  "D": [-0.28340811, 0.07395907, 0.00019259, 0.00000910, 0.0],
  "distortion_model": "plumb_bob"
}
```

Pinhole projection

A 3D point in the camera coordinate frame is

$$\mathbf{X}_c = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}.$$

Normalized image coordinates are

$$x = \frac{X}{Z}, \quad y = \frac{Y}{Z}, \quad r^2 = x^2 + y^2.$$

With intrinsic matrix

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix},$$

ideal pixel coordinates are $u = f_x x + c_x$ and $v = f_y y + c_y$.

Radial and tangential distortion

Radial distortion:

$$x_r = x(1 + k_1r^2 + k_2r^4 + k_3r^6), \quad y_r = y(1 + k_1r^2 + k_2r^4 + k_3r^6).$$

Tangential distortion:

$$x_t = 2p_1xy + p_2(r^2 + 2x^2),$$

$$y_t = p_1(r^2 + 2y^2) + 2p_2xy.$$

Distorted normalized coordinates:

$$x_d = x_r + x_t, \quad y_d = y_r + y_t.$$

Distorted pixel coordinates:

$$u_d = f_x x_d + c_x, \quad v_d = f_y y_d + c_y.$$

Undistortion as inverse mapping

We observe distorted pixels (u_d, v_d) and want ideal pixels (u, v) .

Practical implementation

Instead of solving a closed-form inverse for every point, compute a remapping table:

$$I_{\text{undist}}(u, v) = I_{\text{raw}}(\text{map}_x(u, v), \text{map}_y(u, v)).$$

OpenCV routine

`initUndistortRectifyMap` builds the map; `remap` samples the raw image.

Source: OpenCV camera calibration documentation

Undistortion code path

```
metadata = load_json("saved/metadata/frame_000012.json")
raw = cv2.imread("saved/raw/frame_000012.png", cv2.IMREAD_UNCHANGED)

K = np.array(metadata["K"], dtype=np.float64)
D = np.array(metadata["D"], dtype=np.float64)

new_K, _ = cv2.getOptimalNewCameraMatrix(K, D, image_size, 0.0, image_size)
map_x, map_y = cv2.initUndistortRectifyMap(
    K, D, None, new_K, image_size, cv2.CV_32FC1
)
undistorted = cv2.remap(raw, map_x, map_y, interpolation=cv2.INTER_LINEAR)
```

Browser UI workflow

- 1 Watch the live ROS2 image stream.
- 2 Capture a frame with clear boundary structures.
- 3 Inspect raw image and metadata.
- 4 Click **Apply Calibration / Undistort**.
- 5 Compare raw and undistorted images side by side.

Instructor prompt

Ask students where the geometric change is largest. They should look near the image boundary, not only the center.

Failure recovery plan

Problem	Recovery
Docker missing	Use official Docker installation docs and Codex prompt
Official dataset download too slow	Use default synthetic EuRoC-like bag
Browser shows no frames	Inspect <code>docker compose logs -f</code> and <code>ros2 topic list</code>
Capture fails	Wait for first frame, then refresh browser
Undistortion looks wrong	Check coefficient order: <code>[k1,k2,p1,p2,k3]</code>
Port conflict	Run with <code>WEB_PORT=8081 docker compose up -build</code>

Discussion questions

- ① Why is `/cam0/camera_info` as important as `/cam0/image_raw`?
- ② Why might undistortion crop or warp the image boundary?
- ③ Why is `ros2 bag play` useful for reproducible robotics experiments?
- ④ What changes if this demo is extended to stereo rectification?
- ⑤ What changes if this demo is extended to visual-inertial odometry?

Wrap-up

Part 1

Docker makes the robotics environment reproducible; ROS2 topics and bags make the sensor data stream inspectable and replayable.

Part 2

Calibration parameters become an active image operator: raw distorted frames are remapped into undistorted frames.

Take-home message

In robotics, calibration is not a one-time report. It is part of the runtime data pipeline.

References

- Docker Desktop documentation: <https://docs.docker.com/desktop/>
- ROS2 distributions: <https://docs.ros.org/en/jazzy/Releases.html>
- ROS2 bag tutorial: ROS2 bag tutorial
- ROS2 CameraInfo message: `sensor_msgs/CameraInfo`
- EuRoC MAV dataset: <https://projects.asl.ethz.ch/datasets/euroc-mav/>
- ETH Research Collection EuRoC landing page: <https://doi.org/10.3929/ethz-b-000690084>
- OpenCV camera calibration: OpenCV camera calibration